

Scope e binding

Marco Cerbella

Lezione 7 – 6 ottobre 2026

1 Scope degli identificatori

Definizione. Lo *scope* (ambito) di un identificatore è la parte del programma in cui l'identificatore è significativo, cioè in cui può essere “visto”. Il tipo di scope dipende da dove l'identificatore viene dichiarato.

- **File scope:** l'identificatore è dichiarato fuori da tutti i blocchi e dalle liste di parametri (cioè fuori dalle funzioni). Si può usare dalla dichiarazione fino alla fine del file.
- **Block scope:** l'identificatore è dichiarato dentro un blocco. Si può usare dalla dichiarazione fino alla fine del blocco più piccolo che la contiene (spesso, ma non sempre, il corpo di una funzione).

Esempio.

```
1  int p;                                // file scope
2
3  int fun2(void) {                       // fun2: file scope
4      int r = 3;                         // r, s: block scope
5      int s = r;
6      while (r != s) {
7          r--;
8      }
9      return r;
10 }
```

Osservazione. Non si possono avere due variabili con lo stesso nome nello stesso scope:

```
1  int main() {
2      int a = 5;
3      float a = 6.5;    // errore: a già dichiarata
4  }
```

2 Shadowing

Si può riusare un identificatore in una dichiarazione *annidata* nello scope esistente; la nuova dichiarazione deve avere block scope, in un blocco contenuto in quello esterno. In questo caso la

dichiarazione interna *nasconde* (*shadowing*) quella esterna, che non è visibile nello scope interno.

Esempio.

```
1  int main() {
2      int x = 1, y = 2, z = 3;
3      printf(" x = %d, y = %d, z = %d \n", x, y, z);
4      {
5          int x = 10;
6          float y = 20;
7          printf(" x = %d, y = %f, z = %d \n", x, y, z);
8          {
9              int z = 100;
10             printf(" x = %d, y = %f, z = %d \n", x, y, z);
11         }
12     }
13     return 0;
14 }
```

```
x = 1, y = 2, z = 3
x = 10, y = 20.000000, z = 3
x = 10, y = 20.000000, z = 100
```

Esempio. Qui `x++` modifica la `x` esterna (non è ridichiarata), mentre `y++` modifica la `y` interna; uscendo dal blocco interno la `y` torna a valere 20.

```
1  int main() {
2      {
3          int x = 10, y = 20;
4          {
5              printf("x = %d, y = %d\n", x, y);
6              {
7                  int y = 40;
8                  x++;
9                  y++;
10                 printf("x = %d, y = %d\n", x, y);
11             }
12             printf("x = %d, y = %d\n", x, y);
13         }
14     }
15     return 0;
16 }
```

```
x = 10, y = 20
x = 11, y = 41
x = 11, y = 20
```

Esempio. Fuori dal blocco in cui è dichiarata, una variabile non è accessibile:

```
1  int main() {
2      {
3          int x = 10;
4      }
5      {
6          printf("%d", x);    // errore: x non visibile qui
7      }
8      return 0;
9  }
```

```
error: 'x' undeclared (first use in this function)
```

3 Binding statico

Definizione. Un *binding* è l'associazione tra un nome e l'oggetto che esso denota. Il *binding time* è il momento in cui il binding viene creato (più in generale, il momento in cui viene presa una decisione di implementazione). La regione testuale del programma in cui un binding è attivo è il suo scope.

In C lo scope di un binding è determinato *staticamente*, cioè a tempo di compilazione (*static scoping*), come si vede dal testo del programma e non dall'ordine delle chiamate a run-time. Altri linguaggi usano lo scoping *dinamico*: ad esempio in Perl `my` definisce una variabile locale a scope statico, `local` una a scope dinamico.

4 Esercizio

Esercizio. Qual è l'output del seguente programma?

```
1  #include <stdio.h>
2
3  int f1(void);
4  int f2(int x, int a);
5
6  int a;
7
8  int main() {
9      int a, b, c;
10
11     a = 7;
12     b = f1();
13     c = f2(a, b);
14     printf("%d %d %d\n", a, b, c);
15 }
16
17 int f1(void) {
18     a = 12;
19     printf("%d ", a);
```

```
20     return (a + 5);
21 }
22
23 int f2(int x, int a) {
24     printf("%d ", a);
25     return (x * a);
26 }
```

Soluzione.

- In `main` la `a` locale (che vale 7) nasconde la `a` globale.
- `f1` non dichiara una propria `a`: per binding statico modifica quella *globale*, la pone a 12, stampa 12 e restituisce 17, quindi $b = 17$.
- In `f2(7, 17)` il parametro `a` vale 17: stampa 17 e restituisce $7 \cdot 17 = 119$, quindi $c = 119$.
- Infine `main` stampa 7 17 119.

```
12 17 7 17 119
```